

We have total 8 Wrapper class for 8 primitive data types.

* These wrapper classes have provided the following **final and static fields** to know the size, min value and max value of the corresponding data type.

Example :

```
SIZE    = Will provide size of the given data type
MIN_VALUE = Will provide min value of the given data type
MAX_VALUE = Will provide max value of the given data type
```

Take the example of Byte class :

```
Byte.SIZE = 8 bits
Byte.MIN_VALUE = -128
Byte.MAX_VALUE = 127
```

Here MIN_VALUE, MAX_VALUE and SIZE are the predefined **static and final fields** of Byte class

```
SizeAndRange.java
-----
//Program to find out the range and size of Integral Data type

void main()
{
    IO.println("\n Byte range:");
    IO.println(" min: " + Byte.MIN_VALUE);
    IO.println(" max: " + Byte.MAX_VALUE);
    IO.println(" size :"+Byte.SIZE);

    IO.println("\n Short range:");
    IO.println(" min: " + Short.MIN_VALUE);
    IO.println(" max: " + Short.MAX_VALUE);
    IO.println(" size :"+Short.SIZE);

    IO.println("\n Integer range:");
    IO.println(" min: " + Integer.MIN_VALUE);
    IO.println(" max: " + Integer.MAX_VALUE);
    IO.println(" size :"+Integer.SIZE);

    IO.println("\n Long range:");
    IO.println(" min: " + Long.MIN_VALUE);
    IO.println(" max: " + Long.MAX_VALUE);
    IO.println(" size :"+Long.SIZE);
}
```

Providing _ in numeric Literals :

- * It is available from JDK 1.7V.
- * Using this we can provide _ symbol in large numeric numbers to enhance the readability of the numbers.
- * We cannot start OR end a number with _ symbol
- * Java compiler will ignore this so at the time of execution we cannot see this _ symbol.

```
Literal.java
-----
//We can provide _ in numeric literal [JDK 1.7]

void main()
{
    long mobile = 98_1234_5678L;
    IO.println("Mobile Number is :"+mobile);
}
```

var keyword in java:

From Java 10V, Java introduced the var keyword.

var is used for **local variable only**.

The compiler automatically identifies the datatype based on the assigned value to the variable.

Example:
var x = 12; [x will become int type]

- Points to remember:
- a) Initialization is compulsory in the same line.
 - b) Allowed only for local variables.
 - c) Type is fixed after initialization.

//var keyword [Introduced from java 10]

```
Var.java
-----
void main()
{
    var x = 100;
    IO.println(x);
    x = 900;
    IO.println(x);

    var y = "Java";
    IO.println(y);
    y = "NIT";
    IO.println(y);
}
```

Floating Point Literal :

* If a numeric literal contains **decimal OR fraction** then It is called Floating Point Literal.

Example : 12.90, 78.56, 12.56

* In floating point literal we have 2 data types :

- a) float (32 bits)
- b) double (64 bits)

* Every floating point literal is of type **double** only so, the following statement will generate a compilation error

```
float f = 1.0; //error

We can represent float value explicitly by using these 3 ways :
```

```
float f1 = 0.0F;
float f2 = 0.1f;
float f3 = (float) (0.2 + 1.4 + 9.2);
```

* Even though all the floating point literals are of **double type only** but still java compiler has provided two flavours to represent double value explicitly to enhance the readability of the code.

Example :
double d1 = 12D;
double d2 = 21d;

* A floating point literal we can represent in exponent form.

Example :

double d1 = 15e2;	double d1 = 15e-3;
= 15 X 10 ²	= 15 X 10 ⁻³ ;
= 1500.0	= 0.015;

* An integral literal, We can represent in four different forms i.e decimal, octal, hexa and binary but a floating point literal we can represent in only one **form i.e decimal only**.

Example :
double d1 = 0.199; //It is decimal but not octal becoz it is floating point literal

* An integral literal i.e byte, short, int and long values we can assign to floating point literal directly **but a floating point literal i.e float and double values we cannot assign to integral literal directly**.

Example :
float f = 12L; //VALID

long x = 12.9F; //INVALID

//Programs :

```
Test.java
-----
void main()
{
    float f = 0.0; //Error
    IO.println(f);
}

Test1.java
-----
void main()
{
    float b = 15.29F;
    float c = 15.25f;
    float d = (float) (15.30 + 34.90);
    IO.println(b + " : "+c+" : "+d);
}

Test2.java
-----
void main()
{
    double d = 15.15;
    double e = 15d;
    double f = 90D;
    IO.println(d+" , "+e+" , "+f);
}

Test3.java
-----
void main()
{
    double x = 0129.89;

    double y = 0167;

    double z = 0178; //error

    IO.println(x+" , "+y+" , "+z);
}

Test4.java
-----
void main()
{
    double x = 0X29;

    double y = 0X91.5;

    double z = 0B0.01;

    IO.println(x+" , "+y+" , "+z);
}

Test5.java
-----
void main()
{
    double d1 = 15e-3;
    IO.println("d1 value is :"+d1);

    double d2 = 15e3;
    IO.println("d2 value is :"+d2);
}

Test6.java
-----
void main()
{
    double a = 0791;
    double b = 0791.0;
    double c = 0777;
    double d = 0Xdead;
    double e = 0Xdead.0;
}

Test7.java
-----
void main()
{
    double a = 1.5e3;
    float b = 1.5e3; //E
    float c = 1.5e3F;
    double d = 10;
    int e = 10.5; //E
    long f = 10D; //E
    int g = 10F; //E
    long l = 12.78F; //E
}
```